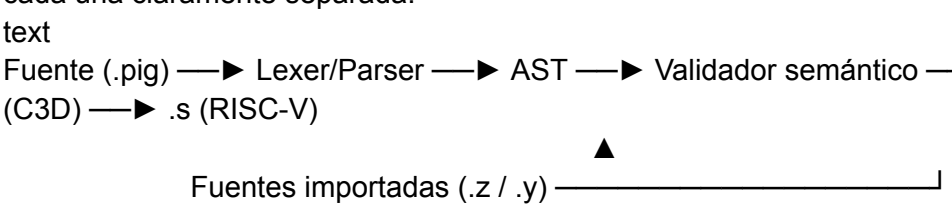
MANUAL TECNICO STACK OVER PIG

1. Resumen del flujo de compilación

El compilador traduce un programa PigLatin a RISC-V RV64I pasando por cuatro etapas, cada una claramente separada:



- Pig (.pig): se analiza y valida con ServicioCompilacionPig.analizar. Su cuerpo se convierte en cuartetas genéricas con GeneradorCuartetasPig.
- Zetariano (.z): cada clase se analiza con ServicioCompilacionZ.analizar. Sus constructores y métodos se convierten en funciones RISC-V (FuncionRiscV) con GeneradorCuartetasZ.
- Y (.y): cada archivo se analiza con ServicioCompilacionY.analizar. Sus funciones se convierten en FuncionRiscV con GeneradorCuartetasY.

Las cuartetas del main de Pig y las FuncionRiscV de Z e Y se entregan a EmisorRiscV.emitir(...), que produce un único archivo .s con:

- sección .data (cadenas, tablas de métodos virtuales),
- sección .bss simulada con .data (heap + heap_ptr),
- _start, main y todas las funciones,
- rutinas de consola (RutinasConsola) necesarias.

---

2. Modelo de memoria

El backend usa tres regiones de memoria, con layout fijo idéntico al que usa el simulador (SimuladorRiscV) y RARS:

Región	Dirección base	Quién la usa
.text	0x00400000	código (main, constructores, métodos, _start, rutinas)
.data	0x10010000	cadenas, .dword, .word, .byte, .ascii
Heap	0x10040000	objetos novus y arreglos novus tipo[n]
Pila	tope 0x7FFFFFF0	variables locales, temporales, structs por valor, marcos de función

- La pila crece hacia abajo (cada addi sp, sp, -framesize reserva).
- El heap crece hacia arriba (bump allocator con heap_ptr).

- Ambas regiones son independientes: no se solapan porque el tope de la pila está muy arriba del heap.

2.1. Qué vive en el heap

Todo lo que se crea con novus en Zetariano, y todo arreglo creado con novus tipo[n]:

- Objetos (Cuenta, Envio, Banco, …): un bloque de 8 * numAtributos bytes. En el offset 0 va un puntero a la tabla de métodos virtuales (vt_Clase), en el offset 8 el primer atributo, en el 16 el segundo, etc.
- Arreglos creados con novus tipo[n]: un bloque de 8 * n bytes, con 8 bytes por elemento (aunque el tipo sea int, double, char o puntero a objeto).
- Arreglos multidimensionales (novus tipo[a][b]): un bloque de a punteros, cada uno apuntando a un bloque de b elementos. Arreglo de punteros (no contiguo).

El heap se reserva con un bump allocator: heap_ptr es una variable global de 64 bits que apunta al primer byte libre. NewObjeto/NewArreglo/NewArregloMulti hacen:
text
heap_ptr += tamaño_del_bloque
dirección_del_objeto = heap + (heap_ptr - tamaño_del_bloque)
No hay free. Nunca se libera memoria (los programas de la práctica son finitos y caben en el heap reservado).

2.2. Qué vive en la pila

Todo lo que no es un objeto ni un arreglo creado dinámicamente:

- variables locales primitivas (int, double, char, bool, String),
- parámetros de funciones/métodos,
- temporales (t0, t1, …),
- structs Y locales por valor (bloque inline),
- arreglos de pila (series x[N] en .pig, int a[N] en .y).

Los parámetros por referencia (objetos Z, {} Struct Y, [] tipo Y) ocupan un slot de 8 bytes en la pila, pero el slot contiene un puntero al objeto/struct/arreglo real (que puede estar en heap o en otra pila).

---

3. Cálculo del frame pointer (s0) y del stack pointer (sp)

Convención adoptada (idéntica en EmisorRiscV y en EmisorRiscVZ):

- sp apunta al último byte reservado del marco (el más bajo en dirección).
- s0 (frame pointer) apunta a un punto fijo del marco: s0 = sp + framesize - 8.
- El marco tiene framesize bytes, siempre múltiplo de 16 (alineación RV64).

3.1. Prólogo (al entrar a una función)

asm
```
addi sp, sp, -framesize       # reserva el marco
sd   s0, framesize-8(sp)      # guarda el s0 anterior (o sea, en s0+0 al final)
sd   ra, framesize-16(sp)     # guarda el return address (en s0-8)
addi s0, sp, framesize-8      # s0 = sp + framesize - 8
```
Por tanto, al final del prólogo:

- 0(s0) = s0 anterior (enlace al marco de arriba),
- -8(s0) = ra (dirección de retorno),
- -16(s0) = primer slot usable para variables, temporales o this.

3.2. Epílogo (al salir)

asm
```
__epilogo_<funcion>:
   ld   ra, -8(s0)
   ld   s0, 0(s0)
   addi sp, sp, framesize
   ret
```
Restaura ra y s0 del marco, libera el marco y vuelve.

3.3. Cálculo de framesize

El emisor emite primero el cuerpo de la función (con un StringBuilder), y durante la emisión va asignando offsets a cada slot nuevo. Al final:
java
```
framesize = ((siguienteOffset + 8 + 15) / 16) * 16;
```
- siguienteOffset es la suma de bytes reservados desde s0 hacia abajo. Empieza en 8 (por el s0 guardado) y crece de 8 en 8 (o de 8*n si es un struct o arreglo inline).
- +8 es porque s0 está en sp + framesize - 8, y el slot más profundo debe quedar dentro del marco (no justo en sp).
- +15) / 16) * 16 redondea hacia arriba a múltiplo de 16, garantizando la alineación que exige RV64 para sp.

Ejemplos verificados (contra los .s generados):
- Cuenta_getSaldo() (solo this, sin locales): siguienteOffset = 16 → framesize = ((16+8+15)/16)*16 = 32. Coincide con el .s (addi sp, sp, -32).
- Cuenta_depositar_decimalis (this + 1 parámetro + varios temporales): framesize = 96.
- main de banco.pig: framesize = 480.
- main de Principal.pig (logística): framesize = 800.

---

4. Layout exacto de un marco de función

Sea N el número de parámetros y framesize el marco calculado:

Offset desde s0	Contenido
+0	s0 anterior (enlace)
-8	ra (return address)
-16	this (solo si la función lo tiene)

-24	parámetro 1 (o this si no hay this, ver §4.1)
-32	parámetro 2
…	…
-24 - (N-1)*8	parámetro N
debajo	variables locales, temporales, structs por valor

Cada slot ocupa 8 bytes, salvo structs por valor que ocupan 8 * numCampos y arreglos inline que ocupan 8 * tamaño.

4.1. Diferencia entre funciones Z y Y

- Funciones Z (con this): baseParam = 1, baseOffset = -24. El primer parámetro va a -24(s0).
- Funciones Y (sin this): baseParam = 0, baseOffset = -16. El primer parámetro va a -16(s0).

Esto se fija en EmisorRiscVZ.emitirFuncion:
java
```
int baseParam  = funcion.tieneThis() ? 1 : 0;
int baseOffset = funcion.tieneThis() ? -24 : -16;
```

4.2. Layout del main de Pig

main no tiene this ni parámetros. Solo reserva framesize bytes para sus variables globales y temporales. Su primer slot está en -16(s0).

---

5. Cómo se accede a un atributo de objeto (layout)

DisposicionObjetos define el layout: atributos del padre primero, luego los del hijo, luego los del nieto, con un ENCABEZADO = 8 reservado al inicio para el puntero a la tabla de métodos virtuales.
text
```
Objeto CuentaCorriente (extiende Cuenta):
  offset   0: puntero a vt_CuentaCorriente
  offset   8: saldo        (heredado de Cuenta)
  offset  16: titular      (heredado de Cuenta)
  offset  24: numero       (heredado de Cuenta)
  offset  32: tipo         (heredado de Cuenta)
  offset  40: movimientos  (heredado de Cuenta)
  offset  48: limite       (propio de CuentaCorriente)
```
Para acceder a obj.campo:
asm

ld   t3, <slot de obj>(s0)   # t3 = puntero al objeto
addi t3, t3, <offset_campo>   # t3 = &obj.campo
ld   t0, 0(t3)   # t0 = obj.campo
El offset del campo se calcula en DisposicionObjetos.offsetCampo(clase, campo): recorre la cadena desde la raíz (cadenaDesdeRaiz) sumando 8 * índice por cada atributo anterior.

---

6. Cómo se accede a una variable local o un temporal

EmisorRiscVZ mantiene un Map<String, Integer> offsets con el offset de cada nombre. Al emitir una carga:
asm
ld   t0, -32(s0)   # slot de la variable/temporal
Al almacenar:
asm
sd   t0, -32(s0)
Cuando el offset no cabe en 12 bits (-2048..2047), se usa un registro auxiliar:
asm
li   t6, <offset>
add t6, s0, t6
ld   t0, 0(t6)
Structs locales por valor: se cargan con addi registro, s0, offset (dirección), no con ld (valor).
Parámetros por referencia (objetos, {}, []): se cargan con ld (el slot contiene el puntero).

---

7. Cómo se pasa un argumento a una función o método

Funciones Z (con this):

- a0 = this
- a1..a7 = argumentos 1..7
- Retorno en a0

Funciones Y (sin this):

- a0..a7 = argumentos 1..8
- Retorno en a0

decimalis se representa internamente como entero de 64 bits escalado $\times 10^6$. Un double de valor 20.5 viaja como 20500000. Por eso double y int viajan en los mismos registros a* (no hay registros f*). La conversión se hace antes de la llamada (adaptarArgumento en Pig, tipoBinario con promoción en Z/Y).

---

8. Cómo se ejecuta un objeto (tabla de métodos virtuales)

Cada clase Z tiene una tabla vt_Clase de 8 * numMétodosVirtuales bytes en .data. Cada objeto guarda en su offset 0 un puntero a su vt_Clase real (el de la clase concreta, no el de la estática).
Al llamar a obj.metodo(...):
asm

```
ld   a0, <slot de obj>(s0)     # a0 = puntero al objeto
ld   t5, 0(a0)                 # t5 = vt_Clase real
ld   t5, <offset_metodo>(t5)   # t5 = dirección del método real
jalr ra, t5, 0                 # salta
```

offset_metodo lo calcula TablaMetodosVirtuales.desplazamiento(etiqueta) y depende de la firma del método (nombre + tipos de los parámetros). Los métodos private no entran a la tabla y se llaman directamente con jal.

---

9. Cómo se ejecuta un arreglo

9.1. Arreglo 1D (novus int[n] en Z, series x[N] en Pig)

- En Z: se reserva un bloque de 8*n bytes en el heap. La variable guarda un puntero al bloque.
- En Pig (arreglo de pila): se reserva un bloque de 8*N bytes en la propia pila. La variable es el bloque (no hay puntero).

Acceso a[i]:

asm

```
# base = puntero o dirección del bloque
ld   t5, <slot de i>(s0)       # t5 = i
ld   t3, <slot de a>(s0)       # t3 = base (puntero si Z, dirección si pila)
slli t5, t5, 3                 # t5 = i * 8
add  t3, t3, t5                # t3 = &a[i]
ld   t0, 0(t3)                 # t0 = a[i]
```

9.2. Arreglo 2D (novus int[a][b] en Z, int m[N][M] en Y)

Arreglo de punteros: un bloque de a punteros a filas, cada fila un bloque de b elementos.

Acceso m[i][j]:

asm

```
# cargar m[i] (puntero a fila)
ld   t5, <slot de i>(s0)
ld   t3, <slot de m>(s0)
slli t5, t5, 3
add  t3, t3, t5
ld   t3, 0(t3)                 # t3 = m[i] (puntero a fila)
# cargar m[i][j]
ld   t5, <slot de j>(s0)
slli t5, t5, 3
add  t3, t3, t5                # t3 = &m[i][j]
ld   t0, 0(t3)                 # t0 = m[i][j]
```

Cuidado con los registros: cuando el índice se calcula con un acceso complejo (por ejemplo, m[f()][g()]), cargar(i) podría pisar el registro de la base. El emisor resuelve esto guardando el índice en un slot temporal de pila antes de calcular la base (direccionElemento en EmisorRiscVZ).

---

10. Cómo se ejecuta una instrucción println / >>

Todos los tipos se convierten a un string terminado en 0 antes de imprimirse:

Tipo	Rutina	Qué hace
textum / cadena	__imprimir_cadena	write(1, cadena, strlen)
numerus / entero	__imprimir_entero	convierte a decimal con signo y hace write
decimalis / flotante	__imprimir_decimal	divide por 10^6, imprime entero, ., 6 decimales, hace write
littera / caracter	__imprimir_caracter	write(1, &char, 1)
bool	concatena con "" → __z_to_string_bool → __concat → __imprimir_cadena	

La concatenación ("a" + b) usa __concat (reserva en el heap con bump allocator).
La lectura (readln, <<) usa __leer_* (syscall read, número 63), que lee del stdin del proceso y deja el resultado en a0. Si el destino es null (solo readln), se descarta.

---

11. Cómo se calcula la dirección de retorno (ra)

ra se guarda en -8(s0) en cada prólogo y se restaura en el epílogo. La llamada se hace con jal ra, etiqueta (guarda pc+4 en ra y salta). El retorno se hace con ret (= jalr x0, ra, 0).
Todas las funciones tienen su propia etiqueta __epilogo_<nombre>, y todas las instrucciones Retorno1 saltan allí. Esto permite tener varios return en la misma función.

---

12. Cómo el simulador interno reproduce esto

SimuladorRiscV es un simulador RV64I escrito en Java, sin dependencias externas. Reproduce fielmente el modelo anterior:

- Inicializa sp = 0x7FFFFFF0 antes de ejecutar.
- memoria es un byte[1 MiB], con BASE_TEXT, BASE_DATA y BASE_STACK mapeados.
- Cada addi sp, sp, -N y sd ra, -8(s0) funciona exactamente igual que en RARS.
- Las syscalls 63, 64, 93 (y 1, 4, 11 de MARS por compatibilidad) se emulan contra System.out y el String stdin.

Consecuencia didáctica: el manual puede afirmar sin ambigüedad que el sp, el s0, el layout de marco, la pila y el heap son idénticos entre el .s emitido, el que se ensambla en RARS y el que ejecuta el simulador propio.

---

13. Cómo se ejecuta una función recursiva

La recursión funciona sin tratamiento especial porque cada llamada hace su propio prólogo:

1. addi sp, sp, -framesize → reserva marco nuevo (baja sp).
2. sd s0, ... / sd ra, ... → guarda enlace al marco anterior.
3. s0 = sp + framesize - 8 → nuevo s0 para esta instancia.
4. Cuerpo con jal a sí misma → nueva instancia con sp más bajo.
5. Epílogo → restaura ra y s0, sube sp.

Las funciones recursivas de Mediciones.y (factorial, fibonacci) funcionan así.

---

14. Cuánto ocupa cada tipo en la pila y en el heap

Tipo	Tamaño en pila	Tamaño en heap (por elemento)
int / numerus	8 bytes	8 bytes
double / decimalis (escalado)	8 bytes	8 bytes
char / littera	8 bytes	8 bytes
boolean / bool	8 bytes	8 bytes
String / textum	8 bytes (puntero a string)	variable (heap)
objeto (Cuenta, Envio…)	8 bytes (puntero)	8 * (1 + numAtributos)
struct Y (Bitacora)	8 * numCampos inline	N/A (siempre en pila)
arreglo T[N]	8 * N inline	8 * N
arreglo T[a][b]	(no aplica en pila)	8*a + a*(8*b)

Todos los slots son múltiplos de 8 para mantener la alineación que exige ld/sd.

Sección de cálculos

1. Cálculo del frame (framesize)

Cada función RISC-V necesita un marco (frame) en la pila donde guardar: s0 anterior, ra de retorno, this (si aplica), los parámetros, las variables locales y los temporales que se van creando durante la emisión.
El tamaño del marco se calcula al final de emitir el cuerpo de la función, porque durante la emisión se van reservando slots nuevos (offsetDe(...)). Se aplica la misma fórmula en EmisorRiscV.emitir(...) (para main) y en EmisorRiscVZ.emitirFuncion(...) (para cada función de Z o Y):
text
framesize = ((siguienteOffset + 8 + 15) / 16) * 16
Qué es siguienteOffset. Es cuántos bytes se han reservado por debajo de s0. Empieza en 8 (el slot de s0 guardado en 0(s0)) y crece de 8 en 8 por cada variable, parámetro, temporal o struct inline nuevo. En EmisorRiscV se consulta con tabla.getSiguienteOffset() (la tabla de Pig ya lleva la cuenta). En EmisorRiscVZ es un campo local (private int siguienteOffset) que arranca en 8 y crece con offsetDe(...).
Por qué los tres sumandos.

- siguienteOffset: bytes ya reservados debajo de s0.
- + 8: porque s0 = sp + framesize - 8. Si no sumamos 8, el slot más profundo (-siguienteOffset(s0)) caería justo en sp, no dentro del marco. Sumar 8 garantiza que el último slot queda dentro.
- + 15) / 16) * 16: redondeo hacia arriba al múltiplo de 16 más cercano. RV64 exige que sp esté alineado a 16 bytes en cada llamada. Sin este redondeo, sp podría quedar desalineado y un sd/ld de 8 bytes en una dirección no alineada a 8 causaría un Load address not aligned en RARS o en el simulador.

Ejemplo numérico verificado (Cuenta_getSaldo, .s real de banco.pig):

- Función con this, sin parámetros, sin locales. siguienteOffset = 16 (8 de s0 guardado + 8 de this).
- framesize = (16 + 8 + 15) / 16 * 16 = 39 / 16 * 16 = 2 * 16 = 32.
- En el .s: addi sp, sp, -32, sd s0, 24(sp), sd ra, 16(sp), addi s0, sp, 24. Correcto.

Ejemplo con varios parámetros y temporales (Cuenta_depositar_decimalis):

- this + 1 parámetro (monto) + temporales internos. siguienteOffset = 88.
- framesize = (88 + 8 + 15) / 16 * 16 = 111 / 16 * 16 = 6 * 16 = 96.
- En el .s: addi sp, sp, -96. Coincide.

Ejemplos del main:

- banco.pig: siguienteOffset = 464 → framesize = (464 + 8 + 15) / 16 * 16 = 487 / 16 * 16 = 30 * 16 = 480.
- Principal.pig (logística): siguienteOffset más grande → framesize = 800.

2. Cálculo del frame pointer (s0) y del stack pointer (sp)

Prólogo (en EmisorRiscV.prologo y EmisorRiscVZ.prologo):

text
```
addi sp, sp, -framesize         # 1. reserva el marco (sp baja)
sd   s0, (framesize - 8)(sp)    # 2. guarda s0 anterior en el tope del marco
sd   ra, (framesize - 16)(sp)   # 3. guarda ra debajo del s0 anterior
addi s0, sp, framesize - 8      # 4. s0 = tope del marco
```
Después del prólogo:

Dirección (relativa a s0)	Contenido
+0(s0)	s0 anterior
-8(s0)	ra (dirección de retorno)
-16(s0)	primer slot utilizable
…	variables, parámetros, temporales
sp	slot más profundo del marco

Por qué s0 = sp + framesize - 8. El marco mide framesize bytes. sp apunta al byte más bajo (el más profundo). s0 debe apuntar al slot más alto del marco, que es el del s0 anterior. Ese slot está 8 bytes por encima del tope: el tope del marco está en sp + framesize, y el slot del s0 anterior está en sp + framesize - 8. De ahí la fórmula.
Por qué s0 y no sp para direccionar variables. Si usáramos sp, cualquier addi sp, sp, -X (por ejemplo al hacer una llamada anidada, o al reservar un slot temporal) cambiaría el offset de todo lo que ya estaba en la pila. Con s0 el offset es estable durante toda la vida del marco, independientemente de lo que pase con sp. Esto es lo que permite la recursión y las llamadas anidadas.
Epílogo (en EmisorRiscV.epilogo y EmisorRiscVZ.epilogo):
text
```
__epilogo_<funcion>:
   ld   ra, -8(s0)        # restaura ra
   ld   s0, 0(s0)         # restaura s0 del llamador
   addi sp, sp, framesize  # libera el marco
   ret                    # jalr x0, ra, 0
```
Después del epílogo, sp y s0 vuelven a los valores que tenían antes del prólogo. El marco desaparece.
Caso framesize > 2048 (solo en EmisorRiscV.main). addi sp, sp, -2048 no cabe en el inmediato de 12 bits con signo de addi. Se sustituye por:
text
```
li   t0, framesize
sub  sp, sp, t0
add  t0, sp, t0
sd   s0, -8(t0)
sd   ra, -16(t0)
addi s0, t0, -8
```

En EmisorRiscVZ no hace falta porque los marcos de funciones Z e Y son pequeños.

3. Cálculo de offsets de parámetros y this

Layout en funciones Z (this + parámetros por referencia):

Slot	Contenido
-16(s0)	this (a0)
-24(s0)	parámetro 1 (a1)
-32(s0)	parámetro 2 (a2)
…	…

Layout en funciones Y (sin this):

Slot	Contenido
-16(s0)	parámetro 1 (a0)
-24(s0)	parámetro 2 (a1)
-32(s0)	parámetro 3 (a2)
…	…

Fórmula en EmisorRiscVZ.emitirFuncion:

```
java
int baseParam  = funcion.tieneThis() ? 1 : 0;
int baseOffset = funcion.tieneThis() ? -24 : -16;
for (int i = 0; i < funcion.parametros().size(); i++) {
   sb.append("sd a").append(baseParam + i).append(", ")
     .append(baseOffset - i * 8).append("(s0)");
}
```

- baseParam = 1 si hay this (los parámetros empiezan en a1, no en a0, porque a0 es this).
- baseOffset = -24 si hay this (el primer parámetro va debajo de this, que ya ocupa -16).
- baseOffset = -16 si no hay this (el primer parámetro va en el primer slot utilizable).

4. Cálculo del tamaño en pila de una variable (tamanoEnBytes)

Cada nombre nuevo que aparece en el cuerpo (una variable local, un temporal) consume un slot. El tamaño depende del tipo:

Tipo Pig	Bytes en pila
numerus, decimalis, littera, bool	8
textum (puntero a cadena en heap)	8
clase Z (Cuenta, Envio, …)	8 (puntero al objeto en heap)
struct Y (Bitacora, Persona, …)	8 * numCampos (inline)
arreglo local (series x[N], int a[])	8 * tamaño (inline)

Fórmula en EmisorRiscVZ.tamanoEnBytes:

java

```
var est = ctx.getTabla().buscarEstructura(tipoPig);
if (est.isPresent()) return 8 * Math.max(1, est.get().atributos().size());

var sim = ctx.getTabla().buscarVariable(nombre);
if (sim.isPresent()) {
    var s = sim.get();
    if (s.dimensiones() > 0 && s.tamanoConocido() != null) {
        return 8 * Math.max(1, s.tamanoConocido());
    }
}

return 8;
```

Los parámetros por referencia (objetos Z, {} Struct Y, [] T Y) también ocupan 8 bytes en pila. Pero el slot contiene un puntero, no los datos. El dato real (objeto, struct, arreglo) está en otro sitio: el heap si es Z, la pila del llamador si es Y.

5. Asignación de un slot nuevo (offsetDe)

Cuando el emisor ve un nombre por primera vez durante la emisión del cuerpo (una variable local, un temporal, un struct local), llama offsetDe(nombre). El método:

1. Comprueba si el slot ya existe (para this y parámetros, reservados antes). Si existe, devuelve el offset.
2. Calcula los bytes con tamanoEnBytes(nombre, tipo).
3. Alinea a 8.
4. Suma esos bytes a siguienteOffset.
5. Devuelve -siguienteOffset (offset negativo desde s0).

java

```
private int offsetDe(String nombre) {
    Integer off = offsets.get(nombre);
    if (off != null) return off;

    String tipo = tiposVariablesActuales.get(nombre);
    int bytes = tamanoEnBytes(nombre, tipo);
    bytes = ((bytes + 7) / 8) * 8;

    siguienteOffset += bytes;
    int nuevo = -siguienteOffset;
    offsets.put(nombre, nuevo);
    return nuevo;
}
```

Ejemplo: una función con this y 2 parámetros empieza con siguienteOffset = 8 + 8(this) + 8(param1) + 8(param2) = 32. Si aparece un temporal nuevo, offsetDe("t0") devuelve -40 y siguienteOffset sube a 40. Si aparece una variable local int x, offsetDe("x") devuelve -48 y siguienteOffset sube a 48. El orden en que aparecen determina el offset final. Lo importante es que el offset devuelto es estable para ese nombre durante toda la función.

6. Cálculo de la dirección de un atributo (offsetCampo)

Dos casos: structs Y (inline en pila) y clases Z (puntero a objeto en heap).

6.1. Struct Y: offset = 8 * índice_campo

EmisorRiscV.calcularOffsetCampo y EmisorRiscVZ.calcularOffsetCampo recorren los atributos del struct en orden y suman 8 por cada uno hasta encontrar el campo:

java

```
var estOpt = tabla.buscarEstructura(tipo);
if (estOpt.isPresent()) {
    int offset = 0;
    for (var entry : estOpt.get().atributos().entrySet()) {
        if (entry.getKey().equals(campo)) return offset;
        offset += 8;
    }
}
```

Ejemplo (Bitacora con campos conteo, montos):

- Bitacora.conteo → offset 0.
- Bitacora.montos → offset 8.
- Bitacora mide 16 bytes en total (2 campos × 8).

6.2. Clase Z: offset = ENCABEZADO + 8 * índice_campo

DisposicionObjetos.offsetCampo recorre la jerarquía desde la raíz hasta la clase, sumando 8 por cada atributo anterior, y añade ENCABEZADO = 8 al final. Los atributos del padre van primero, los del hijo después, los del nieto al final. Eso garantiza que Cuenta.codigo esté en el mismo offset en Cuenta, Urgente e Internacional.

java

```
public int offsetCampo(String clase, String campo) {
   int indice = 0;
   int encontrado = -1;
   for (String c : cadenaDesdeRaiz(clase)) {
      for (String atributo : tabla.buscarClase(c).get().atributos().keySet()) {
         if (atributo.equals(campo)) encontrado = indice;
         indice++;
      }
   }
   return ENCABEZADO + encontrado * BYTES_CAMPO;
}
```

Ejemplo (CuentaCorriente, que hereda de Cuenta):

Offset	Campo	Clase declarante
0	puntero a vt_CuentaCorriente	(encabezado)
8	saldo	Cuenta
16	titular	Cuenta
24	numero	Cuenta
32	tipo	Cuenta
40	movimientos	Cuenta
48	limite	CuentaCorriente

7. Cálculo del tamaño de una clase (tamano)

DisposicionObjetos.tamano(clase) suma los atributos de la clase y de sus ancestros, multiplica por 8 y añade ENCABEZADO = 8 (el puntero a la tabla de métodos virtuales):

java

```
public int tamano(String clase) {
   int campos = 0;
   for (String c : cadenaDesdeRaiz(clase)) {
      campos += tabla.buscarClase(c).get().atributos().size();
   }
   return ENCABEZADO + campos * BYTES_CAMPO;
}
```

Ejemplo:

- Cuenta (5 atributos): 8 + 5*8 = 48.
- CuentaAhorro (5 heredados + 1 propio = 6): 8 + 6*8 = 56.
- CuentaCorriente (5 heredados + 1 propio = 6): 8 + 6*8 = 56.

Esto es lo que reserva emitirNewObjeto en el heap para cada novus.

8. Cálculo de la dirección de un elemento de arreglo

Arreglo 1D: direccion_base + indice * 8. La multiplicación se hace con slli registro, registro, 3 (desplazamiento a la izquierda de 3 = ×8). Después se suma a la base.
asm

```
slli t5, t5, 3        # t5 = indice * 8
add  t3, t3, t5       # t3 = &arreglo[indice]
ld   t0, 0(t3)        # t0 = arreglo[indice]
```

Arreglo 2D (arreglo de punteros): primero se carga el puntero a la fila (arreglo[i]), luego se indexa la fila (fila[j]). El emisor EmisorRiscVZ.direccionElemento lo hace con cuidado: usa t5 para el índice y DIR para la base, y si el índice es complejo (otro AccesoArreglo o un AccesoAtributo), guarda la base en la pila mientras calcula el índice.

9. Cálculo de decimalis (escalado ×10^6)

Un double o float en el fuente se representa internamente como un entero de 64 bits escalado por 10^6. Un decimalis con valor 20.5 viaja como 20500000.
Carga de un literal (cargarLiteral, case "decimalis"):
java

```
long escalado = Math.round(d * 1_000_000L);
```

Aritmética entre decimalis (emitirOperacion):

- +: add directo (ambos ya están escalados).
- -: sub directo.
- *: mul y luego div por 1000000 (porque el producto queda escalado al cuadrado).
- /: mul del dividendo por 1000000 y luego div (para que el resultado quede escalado a 10^6).
- <, >, <=, >=, ==, !=: slt / beq / bne directos (como están escalados, la comparación funciona como enteros).

Impresión (__imprimir_decimal en RutinasConsola): separa parte entera y fracción con remu y divu por 10^6, imprime la parte entera, un ., y 6 decimales rellenos con ceros.
Conversión a string (__z_to_string_double): mismo cálculo, escribe en el heap.
Lectura (__leer_decimal): lee dígitos, detecta el ., cuenta los decimales leídos, multiplica por 10^(6 - decimales_leídos) para normalizar a 6 decimales, y suma parte entera * 10^6 + fracción.

10. Cálculo de etiquetas, saltos y buffers

Todos los contadores de etiquetas, saltos auxiliares y buffers de lectura son globales a la compilación, para que no colisionen entre funciones distintas. Se gestionan en ContextoEmision:
java

```
public int siguienteEtiqueta() { return contadorEtiquetas++; }
public int siguienteSalto()    { return contadorSaltos++; }
public int siguienteBuffer()   { return contadorBuffers++; }
```

Un ContextoEmision es uno por compilación (creado en EmisorRiscV.emitir). Por tanto todas las funciones de Z e Y, y el main, comparten el mismo contador de etiquetas. Si dos funciones usan L0, no colisionan: la segunda usa L1, L2, etc.
Además, ContadorIds.reiniciarEtiquetasGlobales() se llama al inicio de cada compilación (ServicioCompilacionPig.compilarRiscV) para que una compilación no herede etiquetas de una anterior en la misma JVM.

11. Cálculo del marco de un objeto (resumen)

Un novus Cuenta(...):

1. tamaño = 8 + 8 * numAtributos (incluye los heredados).
2. Reserva en el heap: heap_ptr += tamaño, dirección = heap + (heap_ptr - tamaño).
3. En el offset 0 del bloque: sd dirección_vt_Cuenta, 0(objeto).
4. Llama al constructor, que llena los atributos en sus offsets.
5. Guarda el puntero en el slot de la variable local en la pila.

Un novus tipo[n]:

1. tamaño = 8 * n.
2. Reserva en el heap, inicializa a 0 con un bucle corto.
3. Guarda el puntero en el slot de la variable.

Un novus tipo[a][b] (2D, NewArregloMulti):

1. Reserva un bloque de a punteros.
2. Para cada fila, reserva un bloque de b elementos e inicializa a 0.
3. Guarda el puntero al bloque de punteros en la variable.

Resumen ejecutivo de la sección

Los cálculos que gobiernan el backend RISC-V son:

1. framesize = ((siguienteOffset + 8 + 15) / 16) * 16, calculado al final de emitir el cuerpo.
2. s0 = sp + framesize - 8 en el prólogo; offsets negativos desde s0 hacia abajo.
3. Offsets de parámetros: baseParam = tieneThis ? 1 : 0, baseOffset = tieneThis ? -24 : -16.
4. Offsets de atributos: struct Y → 8 * índice; clase Z → 8 + 8 * índice (con herencia padre primero).
5. Direcciones de elementos de arreglo: base + índice * 8 con slli ... 3.
6. decimalis: entero de 64 bits escalado por 10^6; * requiere div por 10^6, / requiere mul por 10^6.
7. Todas las etiquetas son globales (compartidas entre main y funciones) para no colisionar.

Glosario de términos del .s generado

1. Secciones del archivo

.data

Sección de datos inicializados. Contiene:

- cadenas literales (str_0, str_1, …) emitidas con .ascii + .byte 0 (o .string).
- tablas de métodos virtuales (vt_Cuenta, vt_Urgente, …).
- buffers de lectura (buf_lectura_N: .space 256).
- heap estático: heap: .space 65536 y heap_ptr: .dword 0.

Por qué va aquí y no en .bss: RARS no soporta .bss. El emisor lo simula con .data (EmisorRiscV.emitir).

.text

Sección de código. Contiene:

- _start (punto de entrada).
- main (cuerpo del programa Pig).
- funciones de Z (Prueba_constructor, Prueba_metodo, …).
- funciones de Y (suma_numerus_numerus, …).
- rutinas de consola (__imprimir_cadena, __concat, …).

.globl _start

Directiva del ensamblador que hace visible la etiqueta _start al linker/loader. Sin ella, RARS no sabe por dónde empezar a ejecutar. Siempre se emite.

.align 3

Alinea la siguiente etiqueta a 2^3 = 8 bytes. Se usa antes de heap, heap_ptr y vt_* porque ld/sd de 64 bits exigen alineación a 8. Si no está, RARS da Load address not aligned to word boundary.

.space N

Reserva N bytes sin inicializar. Se usa para heap: .space 65536. En RARS cuenta como "zona reservada" en .data.

.dword V

Emite un entero de 64 bits (doubleword). Se usa para heap_ptr: .dword 0.

.ascii "..." / .byte V

.ascii emite una cadena sin terminador. .byte V emite un byte. Combinación usada por el emisor para emitir cadenas con terminador en cero: .ascii "texto" seguido de .byte 0.

.string "..."

Alternativa a .ascii + .byte 0 (añade el terminador automáticamente). El emisor usa .ascii + .byte 0 por control de escape explícito.

2. Etiquetas

_start:

Punto de entrada del programa. Es lo primero que ejecuta RARS al cargar el .s. Su contenido:

```
text
_start:
    <inicialización de tablas vt>
    jal  ra, main
    li   a0, 0
    li   a7, 93
    ecall
```

- Inicializa las tablas de métodos virtuales antes de llamar a main.
- Llama a main con jal.
- Termina con exit(0) (a7 = 93, a0 = 0).

main:

Cuerpo del programa Pig. Tiene su propio marco, su propio s0 y su propio ra.

NombreClase_constructor[_tipos]

Etiqueta del constructor de una clase Z. Ejemplo: CuentaAhorro_constructor_textum_numerus_decimalis. El nombre depende de la firma declarada (tipos de los parámetros), no de los argumentos pasados.

NombreClase_metodo[_tipos]

Etiqueta de un método Z. Ejemplo: Cuenta_depositar_decimalis. Se prefija con la clase declarante (la clase donde está escrito el método), no con la clase del receptor. Eso permite el dispatch virtual.

NombreFuncion[_tipos]

Etiqueta de una función Y. Ejemplo: suma_numerus_numerus. El sufijo son los tipos Pig de los parámetros declarados.

__epilogo_<nombre>

Etiqueta del epílogo de una función Z o Y. Todas las Retorno1 de la función saltan aquí. Contiene las instrucciones de restauración de ra, s0 y sp.

__epilogo_main

Epílogo del main. Mismo contenido, pero con la framesize del main.

L<n>

Etiqueta interna de control de flujo: if, for, while, do-while, saltos de cortocircuito. Se generan con ContextoEmision.siguienteEtiqueta(), un contador global a toda la compilación, para que no colisionen entre funciones.

Larr_ini_<n> / Larr_fin_<n>

Etiquetas del bucle de inicialización a cero de un arreglo nuevo (en NewArreglo). El <n> es único por arreglo.

Lmat_ini_<n> / Lmat_fin_<n> / Lmat_ini_col_<m> / Lmat_fin_col_<m>

Bucle de inicialización a cero de cada fila de un arreglo 2D (NewArregloMulti).

vt_<Clase>

Etiqueta de la tabla de métodos virtuales de una clase Z. Contiene 8 bytes por método virtual, con la dirección de la función real. Se inicializa en _start. El objeto guarda en su offset 0 un puntero a vt_<ClaseReal>.

str_<n>

Etiqueta de una cadena literal en .data. Se emite una vez por cadena única. Si dos métodos usan "hola", ambas usan str_<n>, no se duplica.

buf_lectura_<n>

Buffer de 256 bytes en .data para readln() / << de tipo textum. Cada lectura usa un buffer distinto (contador siguienteBuffer()).

heap: / heap_ptr:

- heap: bloque de 65536 bytes reservado para los objetos y arreglos creados con novus.
- heap_ptr: entero de 64 bits que apunta al primer byte libre del heap. Empieza en 0, se suma a heap para obtener la dirección real.

3. Registros

RISC-V tiene 32 registros x0..x31, con alias convencionales. El emisor usa los alias:

zero (x0)

Registro constante cero. Escribir en él no tiene efecto. Leer de él siempre da 0. Se usa como valor fijo en comparaciones (sltu RES, zero, RES para !=0).

ra (x1, return address)

Dirección de retorno. jal ra, etiqueta guarda pc+4 aquí. El prólogo lo respalda en -8(s0); el epílogo lo restaura y hace ret.

sp (x2, stack pointer)

Puntero al byte más bajo del marco actual. Baja con addi sp, sp, -framesize al entrar y sube con addi sp, sp, framesize al salir.

s0 (x8, frame pointer / saved register)

Puntero al tope del marco actual. Se fija con addi s0, sp, framesize - 8. Todos los slots se direccionan con offsets negativos desde s0, para que sean estables frente a movimientos de sp.

s1-s11 (x9, x18-x27)

Registros "saved": el ABI estándar los preserva entre llamadas. El emisor no los usa (solo s0), para no tener que respaldarlos en cada función.

t0-t6 (x5-x7, x28-x31)

Registros "temporary": el ABI estándar no los preserva entre llamadas. El emisor los usa como registros de trabajo:

- t0 (IZQ): operando izquierdo.
- t1 (DER): operando derecho.
- t2 (RES): resultado.
- t3 (DIR): dirección calculada.
- t4 (IDX): índice.
- t5: registro auxiliar (tabla virtual en jalr, índice en direccionElemento).
- t6 (AUX): registro auxiliar (offsets grandes, constantes).

a0-a7 (x10-x17)

Registros de argumentos y retorno:

- a0: this (en Z) o primer argumento (en Y), y valor de retorno.
- a1-a7: argumentos 1-7.
- a7: número de syscall (64 write, 63 read, 93 exit).
- a0 para syscalls: fd, a1 = buffer, a2 = longitud.

4. Instrucciones usadas

Carga / almacenamiento

- ld rd, off(rs) (load doubleword): carga 8 bytes desde rs+off en rd. Alineación a 8 obligatoria.
- sd rs2, off(rs1) (store doubleword): guarda 8 bytes de rs2 en rs1+off. Alineación a 8 obligatoria.

- lb rd, off(rs) (load byte): carga 1 byte con signo.
- lbu rd, off(rs) (load byte unsigned): carga 1 byte sin signo (0-255). Se usa en __imprimir_cadena, __z_to_string_* para recorrer strings.
- sb rs2, off(rs1) (store byte): guarda 1 byte.
- la rd, etiqueta (load address): pseudo-instrucción. Carga la dirección de la etiqueta (no su contenido). Se expande a auipc + addi.
- li rd, inmediato (load immediate): pseudo-instrucción. Carga una constante en un registro. Se expande a addi (si cabe en 12 bits) o lui + addi.
- mv rd, rs: pseudo-instrucción = addi rd, rs, 0. Copia un registro a otro.

Aritmética

- add rd, rs1, rs2: rd = rs1 + rs2.
- addi rd, rs1, imm: rd = rs1 + imm (imm 12 bits con signo).
- sub rd, rs1, rs2: rd = rs1 - rs2.
- mul rd, rs1, rs2: rd = rs1 * rs2 (producto bajo, 64 bits).
- div rd, rs1, rs2: división con signo.
- divu rd, rs1, rs2: división sin signo. Se usa en __imprimir_entero (después de hacer el valor positivo) y en __z_to_string_*.
- rem rd, rs1, rs2: resto con signo.
- remu rd, rs1, rs2: resto sin signo. Se usa en __imprimir_decimal para separar parte entera y fracción.

Lógica y desplazamiento

- and rd, rs1, rs2: AND bit a bit. Se usa para && (and RES, IZQ, DER).
- or rd, rs1, rs2: OR bit a bit. Se usa para ||.
- xori rd, rs1, imm: XOR con inmediato. Se usa para negar un booleano (xori RES, RES, 1).
- slli rd, rs1, shamt (shift left logical immediate): desplaza a la izquierda. slli X, X, 3 es la forma estándar de multiplicar por 8.
- srli rd, rs1, shamt (shift right logical immediate): desplaza a la derecha sin signo. Se usa para dividir por 8 en NewArreglo.
- srai rd, rs1, shamt (shift right arithmetic immediate): desplaza a la derecha con signo (rellena con el bit de signo).

Comparación

- slt rd, rs1, rs2 (set less than): rd = (rs1 < rs2) ? 1 : 0 con signo.
- sltu rd, rs1, rs2 (set less than unsigned): igual, sin signo.
- sltiu rd, rs1, imm: rd = (rs1 < imm) ? 1 : 0 sin signo. Se usa en == (sltiu RES, RES, 1 es equivalente a RES == 0).
- sltiu rd, rs1, 1: comprobar si rs1 == 0.
- sltu rd, zero, rs1: comprobar si rs1 != 0.

Control de flujo

- beq rs1, rs2, etiqueta (branch if equal): salta si rs1 == rs2.
- bne rs1, rs2, etiqueta (branch if not equal): salta si rs1 != rs2.

- blt rs1, rs2, etiqueta (branch if less than): salta si rs1 < rs2 con signo.
- bge rs1, rs2, etiqueta (branch if greater or equal): salta si rs1 >= rs2 con signo.
- j etiqueta (jump): pseudo-instrucción = jal x0, etiqueta. Salta sin guardar retorno.
- jal rd, etiqueta (jump and link): guarda pc+4 en rd y salta a etiqueta. Se usa con rd=ra para llamadas.
- jalr rd, rs1, off (jump and link register): guarda pc+4 en rd y salta a rs1+off.
- ret: pseudo-instrucción = jalr x0, ra, 0. Vuelve a la dirección de retorno.

Sistema

- ecall (environment call): invoca una syscall del sistema operativo (RARS/simulador). El número va en a7:
    - a7 = 64: write(fd, buf, len) → a0=fd, a1=buf, a2=len. Retorna bytes escritos en a0.
    - a7 = 63: read(fd, buf, len) → a0=fd, a1=buf, a2=len. Retorna bytes leídos en a0.
    - a7 = 93: exit(status) → a0=status. No retorna.

5. Convenciones del código emitido

Prólogo / Epílogo

Cada función emite al entrar:
text
```
addi sp, sp, -framesize         # reserva el marco
sd   s0, (framesize-8)(sp)      # guarda s0 anterior
sd   ra, (framesize-16)(sp)     # guarda ra
addi s0, sp, (framesize-8)      # fija s0
```
Y al salir (en __epilogo_<nombre>):
text
```
ld   ra, -8(s0)               # restaura ra
ld   s0, 0(s0)                # restaura s0 anterior
addi sp, sp, framesize        # libera el marco
ret                           # vuelve a ra
```

framesize

Fórmula:
text
```
framesize = ceil((siguienteOffset + 8 + 15) / 16) * 16
```
- siguienteOffset: bytes reservados debajo de s0.
- +8: porque s0 = sp + framesize - 8.
- Redondeo a 16: alineación obligatoria de sp en RV64.

Layout del marco (offsets desde s0)

Offset	Contenido

0(s0)	s0 anterior
-8(s0)	ra
-16(s0)	this (Z) o primer parámetro (Y)
-24(s0)	primer parámetro (Z) o segundo (Y)
-32(s0)	…

Cada slot ocupa 8 bytes, salvo structs por valor (8 * numCampos) y arreglos inline (8 * tamaño).

Convención de llamada

- Z: a0 = this, a1..a7 = args, retorno en a0.
- Y: a0..a7 = args, retorno en a0.
- Syscalls: a7 = número, resto de args en a0..a2.

Layout de un objeto en el heap

Offset	Contenido
0	puntero a vt_<ClaseReal>
8	primer atributo
16	segundo atributo
…	…

Los atributos del padre van primero, luego los del hijo, luego los del nieto. Garantiza que el offset de un atributo sea el mismo en toda la jerarquía.

Layout de un arreglo en el heap

- 1D: 8 * n bytes, 8 por elemento.
- 2D (NewArregloMulti): arreglo de n punteros a filas; cada fila es un bloque de m * 8 bytes.

decimalis

Representación interna: entero de 64 bits escalado por 10^6. Un 20.5 viaja como 20500000.
Operaciones:

- +, -: directas.

- *: mul y luego div 10^6.
- /: mul 10^6 del dividendo y luego div.
- Comparaciones: directas.

Etiquetas y saltos

- L<n>: contador global a la compilación, para evitar colisiones.
- Larr_ini_<n> / Larr_fin_<n>: por arreglo creado.
- Lmat_ini_<n> / Lmat_fin_<n>: por matriz creada.

Syscalls

- write = 64, read = 63, exit = 93.
- En RARS y en el simulador propio (SimuladorRiscV) los números coinciden.

Rutinas de consola

Cada rutina se emite una vez (aunque varios sitios la usen) y solo si se usó (ctx.getRutinasUsadas()).

Rutina	Qué hace
__imprimir_cadena	write(1, cadena, strlen)
__imprimir_entero	convierte a decimal, write
__imprimir_caracter	write(1, &c, 1)
__imprimir_decimal	divide por 10^6, imprime entero.6 decimales
__leer_caracter	read(0, &c, 1)
__leer_entero	lee dígitos, arma entero con signo
__leer_decimal	lee dígitos, punto, arma escalado $\times 10^6$
__leer_cadena	lee línea completa
__concat	reserva en heap y copia izq + der
__z_to_string_int	entero → string en heap
__z_to_string_char	char → string de 1 carácter

__z_to_string_bool	0/1 → "true"/"false"
__z_to_string_double	decimalis → string

6. Términos de compilación (aparecen en comentarios del .s)

[N] NombreCuarteta

Comentario con el número de cuarteta y su tipo (NewObjeto, LlamadaMetodo1, AsignacionVariable, …). Sirve para rastrear en el .s qué cuarteta generó cada instrucción. Se controla con COMENTAR_CUARTETAS = true en EmisorRiscV.

this

Comentario que marca el sd a0, -16(s0) del prólogo de una función Z.

<nombre>

Comentario del nombre de cada parámetro en el prólogo.

tabla de métodos del objeto

Comentario del sd que escribe el puntero a vt_<ClaseReal> en el offset 0 del objeto nuevo.

&<variable>[0]

Comentario del addi DIR, s0, offset que calcula la dirección base de un arreglo en pila.

&<variable>.<campo>

Comentario del cálculo de dirección de un atributo de struct o de objeto.

<variable>

Comentario del slot cargado/guardado en la pila.

7. Términos de runtime

Heap (heap, heap_ptr)

- Región reservada de 65536 bytes.
- heap_ptr es un entero de 64 bits que apunta al primer byte libre.
- Bump allocator: cada novus reserva y suma al heap_ptr. Nunca se libera.

Stack (sp, s0, marcos)

- Región que crece hacia abajo desde 0x7FFFFFF0 (tope del simulador).

- Cada llamada a función reserva un marco.
- Al retornar, el marco se libera.

Marco (frame)

- Bytes reservados para una función: s0 anterior, ra, this, parámetros, variables locales, temporales.
- Se accede con offsets negativos desde s0.

null (puntero cero)

- Valor 0. Un puntero nulo.
- null == null se traduce a sub t2, t0, t1; sltiu t2, t2, 1 (ambos son 0).
- Un ld 0(t0) con t0 = 0 da Load address not aligned en RARS.

Aliasing

- Dos variables que apuntan al mismo objeto. Ejemplo: alias = internacional.mismo(); (ambos apuntan al mismo bloque del heap).
- Modificar a través de alias cambia el objeto real.
- El compilador no hace nada especial: son punteros, y el hardware los resuelve.

Dispatch virtual

- Mecanismo por el cual obj.metodo() ejecuta la versión del objeto real, no la de la clase estática.
- Cada objeto guarda en su offset 0 un puntero a vt_<ClaseReal>.
- Cada vt_<Clase> tiene 8 bytes por método virtual, con la dirección de la función real.
- Llamada: ld t5, 0(a0); ld t5, off(t5); jalr ra, t5, 0.

Override

- Método de la subclase con la misma firma que el del padre.
- En TablaMetodosVirtuales.construirClase, reemplaza la entrada del padre en la misma posición, apuntando a la función real de la subclase.

Sobrecarga

- Varios métodos con el mismo nombre y distintos tipos de parámetros.
- Se distinguen por el sufijo del nombre RISC-V: Cuenta_costo, Cuenta_costo_numerus, Cuenta_costo_decimalis.
- La resolución se hace en compilación por resolverSobrecarga, con promoción numerus → decimalis.

framesize, siguienteOffset, offset

- siguienteOffset: cuántos bytes reservados debajo de s0.
- framesize: tamaño total del marco (redondeado a 16).
- offset: la posición concreta de un slot (negativo desde s0).

8. Términos del simulador (SimuladorRiscV)

MEMORIA_TOTAL, BASE_TEXT, BASE_DATA, BASE_HEAP, BASE_STACK

Constantes del simulador:

- MEMORIA_TOTAL = 1 MiB (tamaño del byte[] que representa la memoria).
- BASE_TEXT = 0x00400000.
- BASE_DATA = 0x10010000.
- BASE_HEAP = 0x10040000.
- BASE_STACK = 0x7FFFFFF0.

memoria[], registros[], pc

Estado del simulador:

- memoria: byte[1 MiB] que representa todo el espacio de direcciones.
- registros: long[32] con los 32 registros RISC-V.
- pc: contador de programa (dirección de la siguiente instrucción a ejecutar).

Instruccion, Resultado

Tipos internos del simulador:

- Instruccion: una instrucción decodificada (op, rd, rs1, rs2, inmediato, etiqueta).
- Resultado: salida + errores + banderas (salida, error, tiempoAgotado, instrucciones).

Timeout

- Doble: por instrucciones (maxInstrucciones, por defecto 10 millones) y por milisegundos (timeoutMs, por defecto 5 segundos).
- Si se agota, se corta la simulación y se devuelve tiempoAgotado = true.